

Registro de Uso de IA

Rede Turmalina Café

Feito por: Rafael Nagem Volpini

July 31, 2026

1 Momentos de uso

1.1 Momento 1: Ajuste de tema visual (DESIGN.md)

Prompt exato:

Contexto e Persona: Atue como um Engenheiro Front-end Sênior especializado em Design Systems. Vou te passar o arquivo DESIGN.md (exportado do Google Stitch) e preciso que você aplique esse sistema de design ao projeto FastAPI + HTMX + Plotly já existente do dashboard "Rede Turmalina Café" (arquivos: app.py, templates/index.html, templates/loja_detail.html).

Tarefa: leia o DESIGN.md na raiz do projeto e use ele como fonte única de verdade para cores, tipografia, espaçamento, raio de borda e elevação. Substitua os valores hardcoded que já existem no CSS inline de index.html (cores como #5B3E96, #F2A900) pelos tokens definidos no DESIGN.md.

Regras não negociáveis:

1. Vermelho como alerta isolado: o vermelho (#BA1A1A, conforme "Functional Colors" do DESIGN.md) deve aparecer somente em elementos de alerta crítico, loja em prioridade negativa, variação negativa forte. Ele precisa destoar visualmente do resto da paleta, que é dominada por verde água e verde petróleo (#005A54, #00413C). Não deixe o vermelho se misturar como mais uma cor do sistema, ele é o único sinal de urgência da tela.

2. Bordas discretas: não arredonde demais os componentes. Siga a seção "Shapes" do DESIGN.md à risca: 4px (0.25rem) para inputs, botões pequenos e badges não pill; 8px (0.5rem) para cards e containers grandes. Pill (border radius total) só para badges de status e indicadores de ranking, nada mais. Evite qualquer border radius acima de 8px em elementos que não sejam pill.

3. Fundo cinza, não branco: o fundo da página deve usar o tom de cinza claro definido no DESIGN.md (background: #f8f9ff, token surface), não branco puro. Os cards continuam brancos (surface-container-lowest), criando contraste de camada contra o fundo.

4. Gráficos (Plotly), tooltip e legenda como prioridade máxima: configure o hoverlabel de cada go.Figure em app.py pra ser um tooltip totalmente adaptado ao design system, não o padrão cinza do Plotly. Use bgcolor da cor surface-container-lowest, borda na cor primary, fonte Inter em body-md. O texto do tooltip precisa ser claro e indicativo: nunca só o número cru, sempre rótulo completo, unidade e contexto (ex: "Desvio de margem: -4,8% vs. média histórica da loja" ao invés de só "-4.8"). Use hovertemplate para isso, não o hover automático do Plotly.

A legenda de cada gráfico precisa estar sempre visível e bem legível: fonte Inter tamanho label-md do DESIGN.md, contraste alto contra o fundo do card, posicionada de forma que

nunca sobreponha os dados (prefira legenda horizontal acima ou abaixo da área do gráfico, não flutuando por cima das linhas). As cores da legenda precisam ser as mesmas exatas do traço correspondente no gráfico, sem nenhuma variação de tom entre o marcador da legenda e a linha real.

Escopo: só altere CSS, classes, cores e configuração visual dos gráficos Plotly (layout, hoverlabel, legend). Não altere a lógica de cálculo de dados: rotas FastAPI, cálculo de desvio, leitura dos CSVs continuam intocados.

Saída esperada: liste ao final quais arquivos foram alterados e um resumo de 3 a 4 linhas do que mudou em cada um.

O que foi devolvido (resumo): Reescrita de `app/charts.py` trocando Matplotlib/PNG-base64 por Plotly (`go.Figure`), sem build JS/npm (Plotly.js via CDN). Hoverlabel com fundo branco, borda `primary` e fonte Inter em todas as figuras; hovertemplate sempre com rótulo completo, unidade e contexto. Vermelho restrito a alertas reais (queda forte, nota baixa), nunca cor fixa de série. `requirements.txt` trocou matplotlib por plotly. `templates/base.html` ganhou a fonte Inter e o script do Plotly.js. Templates de análise individual e comparativo passaram a incluir o HTML gerado pelo Plotly em vez de imagem estática. `styles/main.css` remapeado inteiro para os tokens do DESIGN.md (cores, raios 4px/8px, pill restrito a badges e ranking, fonte Inter global).

O que foi mantido, alterado ou descartado, com o critério: Mantido integralmente, sem ajuste manual depois. As quatro regras não negociáveis foram checadas contra o resultado antes de aceitar (vermelho isolado, bordas discretas, fundo cinza, tooltip/legenda customizados) e todas bateram, então não houve necessidade de pedir ajuste.

1.2 Momento 2: Nova aba de contexto da rede Turmalina

Prompt exato:

Contexto e Persona: Atue como um Engenheiro Front-end Sênior especializado em dashboards executivos. Preciso adicionar uma terceira aba ao dashboard "Rede Turmalina Café" (FastAPI + HTMX + Plotly), chamada "REDE", ao lado das abas já existentes "PRIORIDADES" e "LOJAS".

Contexto de negócio: essa aba não é acionável como as outras duas (Prioridades aponta as 2 lojas da semana, Lojas mostra o detalhe individual). Essa é a tela de contexto de rede, o retrato que hoje vai pro conselho uma vez por mês, só que recalculado ao vivo a partir dos CSVs em vez de travado num relatório estático.

Tarefa: criar a rota `/rede` em `app.py` e o template correspondente: seletor de período (6 meses / 12 meses / desde a fundação); quatro KPI cards de rede recalculados ao vivo (faturamento bruto, crescimento, margem sobre insumos, ticket médio); contador de quantas das 14 lojas estão acima/abaixo da própria média histórica; gráfico de tendência de expansão mês contra mês anterior; nota fixa deixando claro que é sinal de contexto, não recomendação de expansão; rodapé linkando de volta à aba Prioridades.

Regras de design: mesmos tokens do DESIGN.md já aplicado nas outras abas, sem duplicar CSS.

Escopo: não alterar rotas, templates ou lógica de desvio de Prioridades e Lojas.

Saída esperada: liste arquivos criados/alterados e um resumo de 3 a 4 linhas de cada um.

O que foi devolvido (resumo): `templates/pages/rede.html` criado, reaproveitando classes já existentes. Em `app/data_loader.py`, `resumo_rede()` virou três funções (`rede_kpis()`, `retrato_atual()`, `tendencia_expansao()`). Em `app/charts.py`, novo gráfico de barras divergentes pra tendência. Rota `/visao-geral` virou `/rede` com parâmetro de período. Terceira aba adicionada na

navegação, sidebar removida nessa aba (mesma lógica de Prioridades). Template órfão antigo removido após confirmar que nada mais o referenciava.

O que foi mantido, alterado ou descartado, com o critério: A lógica de cálculo foi mantida integralmente. O componente visual do contador "retrato atual" foi rejeitado e pedido de volta pra refazer: alinhamento, cores e espaçamento não seguiam o padrão dos outros cards. Critério foi consistência visual, não lógica de dado, então só esse componente foi refeito.

1.3 Momento 3: Correção da fórmula do Índice de Prioridade

Prompt exato:

Contexto e Persona: Atue como um Engenheiro de Software Sênior corrigindo a lógica central de um sistema de ranking já existente (dashboard "Rede Turmalina Café", FastAPI + HTMX + Plotly).

Contexto do problema: a função `_indice_prioridade()` hoje usa uma soma de pontos fixos sem critério documentado (margem negativa = 40pt fixo, queda de margem ≥ 3 p.p. = $\min(30, \text{delta} \times 3)$, faturamento abaixo da meta = $\min(20, \text{delta} \times 0,6)$, nota baixa = $(3,5 - \text{nota}) \times 10$, tendência de avaliação em queda = +8pt fixo). Esses números foram inventados sem justificativa e precisam ser substituídos por uma fórmula com critério explícito, decidida agora.

Nova fórmula: (1) margem operacional 30 dias = (faturamento_bruto menos descontos menos custo_insumos menos valor_desperdicio) dividido por faturamento_bruto; (2) variação de margem = margem 30 dias menos média histórica da própria loja, em p.p.; (3) variação de nota = nota média 90 dias menos média histórica da própria loja, em pontos da escala 1 a 5; (4) ranquear as 14 lojas em cada dimensão e converter em percentil 0 a 100 (100 = pior da rede); (5) Índice de Prioridade = $0,6 \times \text{percentil de margem} + 0,4 \times \text{percentil de nota}$; (6) Equipe não entra na fórmula, só aparece como contexto; (7) o "motivo" no card mostra qual dimensão pesou mais, com o valor real e a unidade certa; (8) badge Crítico usa a margem corrigida.

Regras não negociáveis: não inventar constante nova sem justificar; nunca misturar p.p. com % na interface; atualizar todos os textos visíveis pra unidade certa.

Escopo: só `_indice_prioridade()` e as funções de margem que ela usa. Não alterar KPIs de Rede nem tendência de expansão.

Saída esperada: arquivos e funções alterados, antes/depois da fórmula, resumo de 3 a 4 linhas.

O que foi devolvido (resumo): Único arquivo tocado: `app/data_loader.py`. Criadas `_margem_operacional_pct()` e `_margem_historica()` como funções dedicadas (antes a fórmula vivia inline e só descontava `custo_insumos`). `_indice_prioridade()` passou a receber os dois percentis já calculados e devolver só a combinação 0,6/0,4, sem pontos fixos. Nova `_percentil_pior_para_melhor()` rankeia as 14 lojas e converte em percentil, com fallback neutro (50) para valores ausentes. Nova `_motivo_prioridade()` compara os dois percentuais e formata a variação real com a unidade certa. `list_prioridades()` passou a calcular os percentis relativos à rede inteira antes de montar o índice de cada loja, não mais loja a loja isolada.

O que foi mantido, alterado ou descartado, com o critério: A lógica central (percentual 60/40, sem números mágicos) foi mantida. A própria IA sinalizou, sem eu pedir, uma contradição no prompt: pedi "use a mesma conta da aba Rede" mas também escrevi a fórmula com denominador em faturamento bruto, quando a aba Rede na verdade usa faturamento líquido. Ela implementou o que foi escrito literalmente (bruto) e documentou a contradição em vez de decidir sozinha, o que levou à correção do Momento 4.

1.4 Momento 4: Correção do denominador da margem e religação do gráfico de desvio

Prompt exato:

Contexto e Persona: Atue como um Engenheiro de Software Sênior fazendo duas correções de consistência num sistema já existente (dashboard "Rede Turmalina Café", FastAPI + HTMX + Plotly).

Correção 1: `_margem_operacional_pct()` e `_margem_historica()` usam `faturamento_bruto` como denominador; o correto é `faturamento líquido` (`faturamento_bruto` menos descontos), igual à aba Rede. Ajustar as duas funções e tratar divisão por líquido zero ou negativo como margem indefinida, não erro.

Correção 2: `weekly_desvio_margem()` existe mas nenhuma rota a chama; o gráfico "Desvio de Faturamento" nas telas Prioridades e Lojas ainda usa `weekly_desvio_faturamento()`, sobre `faturamento bruto`. Atualizar `weekly_desvio_margem()` com a fórmula corrigida, trocar a chamada nas rotas e templates, e atualizar título, eixo, legenda e `hovertemplate` para "Desvio de Margem Operacional" em p.p.

Regras não negociáveis: uma única fórmula de margem operacional no projeto inteiro depois da correção; listar qualquer lugar que ainda tenha fórmula divergente, mesmo sem uso ativo.

Saída esperada: arquivos e funções alterados, antes/depois de cada fórmula, confirmação de fórmula divergente remanescente.

O que foi devolvido (resumo): `_margem_operacional_pct()` passou a dividir por `faturamento líquido`, retornando NaN ("margem indefinida") quando líquido ≤ 0 . Guard adicionado em `_status_margem()` pra não deixar NaN cair silenciosamente em "Estável". `store_metrics()` e `list_stores_summary()` passaram a tratar margem indefinida como None em vez de arredondar erro, com None tratado como pior caso na ordenação. `weekly_desvio_margem()` passou a usar a mesma fórmula líquida via `_margem_historica()`. Em `app/charts.py`, `desvio_faturamento()` virou `desvio_margem()`, com eixo, `hovertemplate` e linha de referência todos trocados de % para p.p. Rotas e templates atualizados pra chamar a função nova, com guards de "None" nos valores exibidos. Dois resíduos identificados e não tocados por estarem fora do escopo: a coluna `margem_pct` antiga em `load_vendas()` (morta, sem leitura) e `rede_kpis()` com implementação própria da mesma fórmula (duplicada, não compartilhada).

O que foi mantido, alterado ou descartado, com o critério: A correção foi mantida integralmente. Os dois resíduos foram levados de volta pra decisão: optei por limpar os dois numa tarefa seguinte, confirmando antes que os 4 KPIs de rede não mudavam de valor com a refatoração (testado e confirmado: R\$ 20.840.469, +26,5%, 67,7%, R\$ 25,19, idênticos antes e depois).

1.5 Momento 5: Contador com retrato de agora e média do período

Prompt exato (versão final, após ajuste sobre uma primeira rodada que substituiu em vez de complementar):

Contexto e Persona: Atue como um Engenheiro de Software Sênior ajustando um componente que você mesmo acabou de implementar no dashboard "Rede Turmalina Café".

Contexto: o card "X lojas indo bem / Y lojas indo mal" foi alterado numa tarefa anterior para mostrar a média do período selecionado, substituindo o retrato do mês mais recente. Isso foi longe demais: o retrato de agora deveria ter sido mantido ao lado da média, não substituído.

Tarefa: reintroduzir o retrato de agora (contagem inteira do mês mais recente, rótulo "Retrato de agora (mês/ano)", não varia com o período) mantendo a média do período já

implementada (uma casa decimal, rótulo "Média do período selecionado"), lado a lado ou em cards adjacentes.

Regras não negociáveis: retrato de agora não varia entre os três períodos; média varia. Não alterar `tendencia_expansao()` nem lógica de Prioridades/Lojas.

Saída esperada: arquivos alterados, os dois números para os 3 períodos, confirmando o comportamento esperado de cada um.

O que foi devolvido (resumo): `retrato_atual()` reescrita pra devolver as duas leituras juntas: contagem inteira do último mês (`agora_acima/agora_abaixo`, via `.iloc[-1]`) e a média por mês já implementada antes (`media_acima/media_abaixo`), sem alterar o cálculo da média em si. Novo dicionário só pra formatar o rótulo do mês por extenso. O card único virou dois cards lado a lado no mesmo padrão de grid já usado na página. Confirmado com dado real: retrato de agora ficou fixo em 9 indo bem / 5 indo mal nos três períodos (6m, 12m, fundação), como esperado; a média variou de fato entre eles (8,4/4,3 em 6 meses, 6,2/4,4 em 12 meses e desde a fundação).

O que foi mantido, alterado ou descartado, com o critério: Mantido integralmente. Uma observação registrada à parte: "12 meses" e "desde a fundação" deram o mesmo número porque o CSV só tem 12 meses de histórico disponível (a empresa foi fundada em 2021, mas o dado começa em 07/2025), não por erro de cálculo.

1.6 Momento 6: Semanas incompletas nos gráficos de desvio semanal

Prompt exato:

Contexto e Persona: Atue como um Engenheiro de Software Sênior corrigindo um problema de honestidade visual num gráfico já existente.

Contexto, já diagnosticado numa tarefa anterior de investigação: `weekly_desvio_margem()` agrupa por semana ISO sem checar completude. Quando a série termina no meio de uma semana, o último ponto do gráfico "Desvio de Margem Operacional" pode ter só 2 dos 7 dias, muito mais ruidoso que os pontos anteriores.

Tarefa: excluir semanas incompletas do gráfico. Contar dias distintos de dado por semana e manter só as semanas com 7 dias completos, sem forçar sempre 8 pontos se não houver 8 semanas completas disponíveis. Aplicar o mesmo princípio em qualquer outro gráfico semanal do sistema.

Regras não negociáveis: não completar semana incompleta com dado inventado; não alterar lógica de margem, desvio, referência histórica, Índice de Prioridade ou KPIs de Rede, só a filtragem de quais semanas entram no gráfico.

Saída esperada: arquivos e funções alterados, quantas semanas completas existem para pelo menos duas lojas de teste, antes/depois dos pontos plotados.

O que foi devolvido (resumo): `weekly_desvio_margem()` passou a contar `nunique()` de datas por semana e manter só semanas com exatamente 7 dias, antes de agregar. Confirmado com dado real: Turmalina Praia do Canto tem 43 de 53 semanas completas na série inteira, Turmalina Belvedere 11 de 16. Antes da correção, o último ponto de ambas era uma semana de só 2 dias (29 e 30/06), com desvio de -0,30 p.p. e -0,76 p.p. respectivamente; depois da correção, o último ponto passou a ser a última semana completa (22/06), com valores bem diferentes (+0,48 p.p. e -0,80 p.p.), confirmando que o ponto truncado antigo era majoritariamente ruído. Rotas testadas depois da mudança, todas retornando HTTP 200.

O que foi mantido, alterado ou descartado, com o critério: Mantido integralmente. Um desvio da instrução literal ocorreu e está documentado como Recusa 2, abaixo.

1.7 Momento 7: Eixo compartilhado nos gráficos comparativos e correção de tooltip

Prompt exato (resumido, duas rodadas: diagnóstico e correção do gráfico de margem, depois extensão para os outros dois comparativos):

Contexto e Persona: Atue como um Engenheiro de Software Sênior corrigindo um bug de eixo num gráfico comparativo já existente. Contexto: no gráfico "Desvio de Margem Operacional" da tela de comparativo, o eixo X está fora de ordem cronológica. Causa provável: a correção de semanas incompletas (Momento 6) foi aplicada por loja, independentemente, então lojas com gaps diferentes acabam comparando semanas calendário diferentes uma da outra.

Tarefa: eixo X compartilhado e cronologicamente correto pras duas lojas, com `merge(how="left")` e gap real (sem inventar dado) onde uma loja não tiver semana completa na janela compartilhada. Corrigir também o posicionamento do tooltip, que tampava o próprio dado, aplicando hovermode unificado em todos os gráficos do sistema, não só neste.

(Segunda rodada, depois de confirmado o padrão): aplicar exatamente a mesma correção de eixo compartilhado aos outros dois gráficos comparativos que tinham o mesmo problema (avaliações e faturamento mensal), identificados pela própria IA na primeira resposta como fora do escopo original mas com o mesmo risco.

O que foi devolvido (resumo): Nova `weekly_desvio_margem_comparativo()` com eixo fixo de semanas civis compartilhado (`merge how="left"`, gaps reais via `NaN`, `connectgaps=False`). `hovermode="x unified"` aplicado globalmente em `_apply_layout()`, removendo prefixo de data duplicado dos hovertemplates. Diagnóstico confirmou a causa: LJ09 usava suas 8 semanas mais recentes, LJ14 tinha que buscar mais fundo (até 13/04) por causa de mais gaps, gerando eixo fora de ordem. Depois da correção, eixo único 11/05 a 29/06 pras duas, com gaps reais nas semanas sem dado completo. Numa segunda rodada, o mesmo padrão (`weekly_avaliacoes_comparativo()`, `monthly_faturamento_comparativo()`) foi aplicado aos outros dois gráficos comparativos, revelando inclusive um caso real de gap esperado (Belvedere abriu em março/2026, aparecendo sem dado nos 8 meses anteriores no comparativo de faturamento).

O que foi mantido, alterado ou descartado, com o critério: Mantido integralmente nas duas rodadas. A IA sinalizou, sem eu pedir, que o mesmo bug existia em dois outros gráficos fora do escopo da tarefa original, e não mexeu neles até eu confirmar que queria a correção estendida, o que preservou o escopo pedido em cada prompt em vez de fazer mudança não solicitada.

1.8 Momento 8: Diagnóstico e correção do bug de arredondamento no ranking

Prompt exato (duas rodadas: diagnóstico, depois correção):

Contexto e Persona: Atue como um Engenheiro de Software Sênior investigando uma divergência entre o ranking do Índice de Prioridade que o app mostra e um recálculo independente feito fora do sistema, direto dos CSVs.

Refiz o cálculo do Índice de Prioridade em Python puro, fora do app, com minha própria metodologia (janela de 30/90 dias terminando na data mais recente da rede, percentil ordinal), e meu resultado (LJ13, LJ08, LJ14, LJ09 no topo) diverge do que a tela mostra (LJ09, LJ14). Compare minha metodologia contra o código real, função por função, e aponte onde a diferença está: janela de data, critério de desempate, ou filtro de loja que eu não apliquei. Não corrija nada ainda, só diagnostique.

(Segunda rodada, depois do diagnóstico): corrija dois problemas confirmados: (1) `store_metrics()` arredonda a variação de margem antes de `list_prioridades()` usar esse valor pra rankear, criando empates artificiais que a variação de nota (não arredondada) não tem; (2) o percentil ordinal amplifica variação pequena com só 14 lojas na amostra. Troque por percentil de magnitude: a proximidade da queda real da própria loja a um limiar de severidade (3 p.p.

de margem, já existente; um limiar novo de 0,5 ponto de nota, documentado como premissa), não a posição entre as lojas.

O que foi devolvido (resumo): No diagnóstico, a IA confirmou que a causa não era janela de data nem filtro de loja (minhas duas primeiras suspeitas), e sim o arredondamento prematuro: cinco lojas com variações de margem reais entre -0,17 e -0,24 p.p. arredondavam todas para "-0,2" antes de entrar no `rank()`, e o desempate por média de posição as nivelava artificialmente, inflando o índice da LJ09 às custas da LJ13 e LJ08 (as piores de verdade em margem). Na correção, `store_metrics()` passou a manter `variacao_margem_pp_raw` sem arredondar (mesmo padrão já usado pra nota); `_percentil_pior_para_melhor()` foi substituída por `_percentil_por_magnitude()`, medindo a distância da queda real até `MARGEM_QUEDA_CRITICA_PP=3,0` e a nova `NOTA_QUEDA_CRITICA_PONTOS=0,5`; variação positiva ou zero vira percentil 0. Resultado recalculado para as 14 lojas: as prioritárias mudaram de LJ09/LJ14 para LJ09/LJ04, e 5 lojas (LJ02, LJ03, LJ05, LJ07, LJ10) ficaram com índice exatamente 0, por não terem queda real em nenhuma dimensão.

O que foi mantido, alterado ou descartado, com o critério: Mantido integralmente nas duas rodadas. O diagnóstico rejeitou corretamente duas das minhas três suspeitas (janela de data, filtro de loja) antes de confirmar a real (arredondamento), evitando uma correção no lugar errado.

1.9 Momento 9: Fechando três lacunas entre documento e sistema (mix de produtos, tempo de espera, eficiência de equipe)

Prompt exato:

Contexto e Persona: Atue como um Engenheiro de Software Sênior fechando três lacunas entre o que o documento de projeto promete e o que o sistema hoje entrega, num dashboard já existente (Rede Turmalina Café, FastAPI + Jinja2 + HTMX + Plotly).

Contexto: três coisas foram documentadas como parte do fluxo de investigação do sistema, mas nunca ligadas em nenhuma tela: (1) mix de produtos, pra explicar qual categoria está puxando a margem pra baixo quando Margem é o motivo de prioridade de uma loja; (2) tempo médio de espera, como sinal de Cliente quando não há comentário explicando a nota; (3) eficiência de equipe, como sinal auxiliar do ranking, sem pesar no índice. As três já têm dado carregado e tratado no código (`load_itens()`, `tempo_espera_medio` em `store_metrics()`, `horas_trabalhadas_equipe` em `load_vendas()`), só falta exibir.

Tarefa 1: mix de produtos como detalhe de margem, exibido quando o motivo de prioridade da loja for margem, com a categoria de pior margem (`turmalina_itens.csv`).

Tarefa 2: tempo de espera como fallback de Cliente, exibido quando não houver comentário de atenção, com limiar de referência documentado (8 minutos).

Tarefa 3: card de Equipe como contexto auxiliar (faturamento por hora trabalhada vs. histórico), sempre visível, sem entrar no Índice de Prioridade.

Regras não negociáveis: nenhuma das três tarefas altera o Índice de Prioridade, os percentis ou qualquer cálculo de margem/nota; nada de dado inventado onde não existe, mostrar mensagem de ausência; reaproveitar os dados já carregados, não recriar.

Saída esperada: arquivos e funções alterados, confirmação com o par de lojas prioritárias atual do que aparece em cada um dos três elementos.

O que foi devolvido (resumo): Em `app/data_loader.py`: `load_itens()` ganhou normalização de `id_loja` (mesma convenção dos outros loaders); `pior_categoria_margem(id_loja)` (nova) agrega margem por categoria no mês mais recente; `eficiencia_equipe(id_loja)` (nova) calcula faturamento por hora trabalhada, 30d vs. histórico, sem percentil nem peso no índice; `TEMPO_ESPERA_LIMIAR_MIN = 8,0` (nova constante, documentada); `store_metrics()` passou a

devolver o limiar e se está acima dele; `list_prioridades()` anexa os três elementos só nos 2 cards críticos, sem tocar em índice, percentis ou ranking. Testado com o par atual (Praia do Canto/LJ09, BH Shopping/LJ04): ambas com motivo "nota", então o bloco de mix de produtos não aparece para nenhuma das duas (testado corretamente à parte para lojas com motivo margem); tempo de espera também não aparece (ambas já têm comentários de atenção suficientes); eficiência de equipe aparece sempre, LJ09 R\$56,21/h (+4,2% vs. histórico), LJ04 R\$144,22/h (+10,8% vs. histórico).

O que foi mantido, alterado ou descartado, com o critério: Mantido integralmente. Uma limitação real ficou registrada: como o mix de produtos e o tempo de espera são condicionais, e as 2 lojas prioritárias do momento não disparam nenhuma das duas condições, essas duas funcionalidades não apareciam em lugar nenhum navegável do sistema logo após esta tarefa (resolvido no Momento 10).

1.10 Momento 10: Estendendo o mix de produtos para a tela Análise Individual

Prompt exato:

Contexto e Persona: Atue como um Engenheiro de Software Sênior estendendo uma funcionalidade já validada, mas hoje invisível na prática, num dashboard já existente (Rede Turmalina Café, FastAPI + Jinja2 + HTMX + Plotly).

Contexto: `pior_categoria_margem()` foi implementada numa tarefa anterior e validada isoladamente, mas só está ligada nos 2 cards de loja crítica da tela Prioridades, condicionada a `motivo_dimensao == "margem"`. Como as 2 lojas prioritárias de hoje têm motivo "nota", esse bloco não aparece em nenhum lugar navegável do app no momento, mesmo a lógica estando correta.

Tarefa: exibir o mix de produtos também na tela Análise Individual (Lojas), sempre visível, para qualquer loja, não condicionado a ser prioritária nem ao motivo, reaproveitando `pior_categoria_margem()` sem duplicar. Manter o bloco em Prioridades como está.

Saída esperada: arquivos alterados, confirmação testando de verdade na tela (não só a função isolada), qual categoria aparece para pelo menos 3 lojas diferentes com dado, e uma loja sem dado.

O que foi devolvido (resumo): `app/routes.py: analise_individual()` passou a incluir `mix_pior_categoria: dl.pior_categoria_margem(id_loja)` sempre, sem condição de motivo ou prioridade. Novo card em `templates/pages/analise_individual.html`, reaproveitando o mesmo estilo visual já usado em Prioridades. Testado de verdade na tela (via requisição HTTP renderizada, não só a função isolada, que era o problema anterior): LJ01 Turmalina Savassi (Grãos E Varejo, 33,3%), LJ04 Turmalina BH Shopping (Varejo, 31,8%), LJ09 Turmalina Praia do Canto (Grãos E Varejo, 36,0%), todas retornando categoria e margem reais. Nenhuma das 14 lojas reais está sem registro em `turmalina_itens.csv` hoje, então o caso "sem dado" foi validado só chamando a função com um id inexistente (retorna `None`, mensagem de ausência confirmada no código, mas nenhuma loja real aciona esse caminho hoje).

O que foi mantido, alterado ou descartado, com o critério: Mantido integralmente. Essa tarefa corrigiu uma lacuna que eu mesmo relatei errado numa resposta anterior (achei que já estava visível em Análise Individual quando na verdade só tinha sido validada isoladamente, não testada na tela) — vale registrar que a checagem cuidadosa desta vez (teste na tela renderizada, não só na função) evitou repetir o mesmo erro de relato.

1.11 Momento 11: Investigação e correção de id_loja malformatado em turmalina_itens.csv

Prompt exato:

Contexto e Persona: Atue como um Engenheiro de Dados Sênior investigando uma possível duplicidade de dado num sistema já existente (Rede Turmalina Café, FastAPI + Jinja2 + HTMX + Plotly).

Contexto: ao testar `pior_categoria_margem()` para todas as 14 lojas, foi reportado que `turmalina_itens.csv` tem linhas para "LJ13" e também para uma variante malformatada "LJ-13" (com hífen).

Tarefa: confirmar a contagem real de linhas de cada variante; confirmar se `load_itens()` normaliza `id_loja` da mesma forma que os outros 3 loaders; se não normalizar, aplicar a mesma `normalize_id_loja()` já usada nos outros, e recalculando `pior_categoria_margem("LJ13")` antes e depois; confirmar se o mesmo problema existe nos outros 3 CSVs.

Regras não negociáveis: não alterar `pior_categoria_margem()` nem a fórmula de margem de produto, só a normalização de entrada; não alterar nenhuma outra lógica do sistema; listar toda variante malformatada encontrada, não só a primeira.

Saída esperada: contagem exata antes da correção, causa raiz, resultado antes/depois, confirmação sobre os outros 3 CSVs.

O que foi devolvido (resumo): Contagem real antes da correção: 60 linhas com "LJ13", 45 linhas com "LJ-13" (total real da loja: 105, só 60 visíveis). Causa raiz: `normalize_id_loja()` já existia e era chamada por `load_itens()`, mas só fazia `strip().upper()`, sem remover hífen, então "LJ-13" continuava sendo tratado como uma loja diferente e inexistente. Correção: a função compartilhada (usada pelos 4 loaders) passou a remover hífen também. `pior_categoria_margem("LJ13")` antes: 60 linhas, categoria "Grãos E Varejo", 32,3% de margem; depois: 105 linhas, mesma categoria e mesmo percentual (coincidência de dados, não erro na correção, o breakdown completo por categoria mudou de fato, só o "pior" continuou o mesmo). Confirmado que os outros 3 CSVs não têm essa variante malformatada (só `turmalina_avaliacoes.csv` tinha `id_loja` minúsculo, já corrigido em correção anterior). Índice de Prioridade recalculado depois da correção: idêntico (LJ09 34,5, LJ04 33,9), confirmando que a correção não vazou pra nenhum outro cálculo.

O que foi mantido, alterado ou descartado, com o critério: Mantido integralmente. A própria investigação revelou, de passagem, uma segunda lacuna de normalização (categoria de produto sem dicionário de sinônimos, causando "Grãos E Varejo" vs. "Graos E Varejo" como categorias distintas), levada para decisão separada, ainda em aberto no momento deste registro.

2 Desacordo

Situação: pra decidir o sinal de Expansão, a primeira ideia foi somar o desvio positivo de faturamento de cada loja e comparar com a soma dos desvios negativos, sempre comparando cada loja contra a própria média histórica completa.

Contestação: essa conta tem o mesmo problema que já apareceu na Seção 1 do documento de projeto: como a rede inteira está crescendo, comparar qualquer período recente contra o histórico completo (que inclui os meses mais fracos do início) tende a dar resultado positivo, não importa se a loja está indo bem ou mal agora. Rodei essa conta com o dado real e deu 14 de 14 lojas positivas, um resultado que não diz nada, é só efeito do crescimento geral da rede.

Decisão que prevaleceu: em vez de comparar contra o histórico inteiro, comparar cada mês contra o mês anterior. Rodando isso nos últimos 3 meses, o resultado variou de verdade: 7 positivas e 7 negativas num mês, 14 e 0 no mês seguinte, 9 e 5 no mês depois. Essa variação mostra que a conta reflete o que está acontecendo de fato, não é só reflexo do crescimento geral.

3 Recusa

3.1 Recusa 1: Stack React, Tailwind CSS e shadcn/ui

Proposta: protótipo do dashboard em React, Tailwind CSS e shadcn/ui, com gráficos em Recharts.

O que havia de errado ou insuficiente: exigiria build step (npm install, bundler), o que aumenta o risco de o protótipo não rodar em outra máquina a partir de um README de uma página, requisito obrigatório do case. Foi usado só como exploração visual (wireframe de alta fidelidade), não como stack final.

O que foi usado no lugar: FastAPI + Uvicorn + Jinja2 + HTMX + Plotly, sem build step, reproduzível com `pip install` e um comando.

3.2 Recusa 2: Regra literal de semana completa quebraria o gráfico de avaliações

Proposta: no Momento 6, a instrução pedia, ao pé da letra, "contar dias distintos de venda" como critério de semana completa, e aplicar o mesmo princípio em qualquer outro gráfico semanal do sistema, incluindo o de avaliações de clientes.

O que havia de errado ou insuficiente: avaliação é um evento esparsos, não diário como venda. Testando a regra literal antes de aplicar, a IA identificou que nenhuma das 24 semanas com avaliação da Turmalina Praia do Canto tinha 7 dias distintos com avaliação, porque clientes não avaliam todo santo dia. Aplicar a regra ao pé da letra deixaria o gráfico de avaliações permanentemente vazio.

O que foi usado no lugar: um critério equivalente adaptado para dados esparsos: a semana civil precisa ter decorrido inteira dentro do período coberto pela base (não ultrapassar a data mais recente disponível), sem exigir avaliação em todos os 7 dias. Isso derrubou só a semana genuinamente truncada (1 caso na base inteira, um bucket de 29/06 na Turmalina BH Shopping), preservando o resto do gráfico.